Task 1 Report — Data Analysis & Preprocessing

Project: Fraud detection across e-commerce (`Fraud_Data.csv`) and bank credit-card (`creditcard.csv`) transactions, enriched with IP-based geolocation.

Author: Yostina Abera **Date:** 2026-06-06 **Email:** bet30539@gmail.com **Scope:** Cleaning, EDA, geolocation integration, feature engineering, and class-imbalance strategy. Modeling and explainability follow in Tasks 2–3.

---

1. Executive summary

Two labelled fraud datasets were cleaned, explored, and prepared for modeling. Both are severely imbalanced (**9.4%** fraud in e-commerce, **0.17%** in credit-card), which dictates every downstream choice: accuracy is discarded in favour of **AUC-PR / recall / precision / F1**, and resampling (**SMOTE**) is applied to the **training fold only**.

The headline analytical finding is that fraud in the e-commerce data is overwhelmingly **automated and device-centric**, not demographic. The two engineered features that separate fraud most cleanly are:

- `time_since_signup` — legitimate users buy a median of **1,443 hours** after signup; fraudulent purchases occur a median of **0 hours** after signup, with **53.7%** of fraud landing within the first hour.
- `device_user_count` — fraudulent transactions sit on devices shared by an average of **7.15 distinct accounts**, versus **1.12** for legitimate ones.

Raw fields (`purchase_value`, `age`, `source`, `browser`, `sex`) carry almost no discriminative signal on their own. The value is created in feature engineering.

---

2. Data cleaning and preprocessing

All cleaning logic lives in tested modules (`src/cleaning.py`, `src/data_loader.py`) so the steps are reproducible and verifiable.

2.1 Fraud_Data (151,112 rows)

Step	Action	Result
Data types	Parsed `signup_time`/`purchase_time` to `datetime`; cast `source/browser/sex` to `category`; kept `ip_address` numeric for range lookup	Timestamps and categoricals now usable for feature engineering
Missing values	Audited all columns	**None** — no imputation required
Exact duplicates	`drop_duplicates()`	**0** removed

Step	Action	Result
Shared devices	Inspected, **retained**	19,331 rows share a `device_id` with another row — *deliberately kept* because device sharing is a fraud signal, not noise

2.2 creditcard (284,807 rows)

Step	Action	Result
Missing values	Audited	**None**
Exact duplicates	`drop_duplicates()`	**1,081** removed → 283,726 rows
Types	`Time`, `Amount`, `V1–V28` already numeric (PCA)	No change needed

2.3 Documented missing-value policy

Although neither dataset currently has nulls, `handle_missing_values()` encodes a defensible policy for future / re-pulled data: **drop columns >50% missing**, **median-impute** numeric (robust to skew), **mode-impute** categorical. This is a guardrail, applied automatically but inert on the current data.

---

3. Exploratory data analysis

3.1 Class imbalance (the defining constraint)

Dataset	Legitimate	Fraud	Fraud rate
Fraud_Data	136,961	14,151	**9.36%**
creditcard	283,253	473	**0.17%**

A trivial "always legitimate" classifier would score **90.6%** / **99.83%** accuracy while catching zero fraud — so accuracy is abandoned. We evaluate with **AUC-PR** (primary), **recall** (catch fraud), **precision** (limit false alarms) and **F1**. *Figures: `fraud_class_balance.png`, `cc_class_balance.png` (log scale).*

3.2 Univariate distributions

- **`purchase_value`** — right-skewed, median **$35** (range $9–$154).
- **`age`** — roughly normal, median **33** (range 18–76).
- **`Amount`** (credit-card) — heavily right-skewed; requires scaling.
- Categorical volume is well spread across `source`, `browser`, `sex`. *Figures: `fraud_univariate_numeric.png`, `fraud_univariate_categorical.png`, `cc_univariate.png`.*

3.3 Bivariate relationships with the target

Raw features barely move with the target — an important negative result:

Feature	Fraud-rate spread across levels
`source`	8.9% (SEO) → 10.5% (Direct)
`browser`	8.7% (IE) → 9.9% (Chrome)
`sex`	9.1% (F) → 9.6% (M)

`purchase_value` and `age` distributions are nearly identical across classes. **Conclusion:** fraud is not explained by demographics or the purchase itself — it lives in *behavioural/temporal* patterns, which motivates the engineered features below. *Figures:* *`fraud_bivariate_numeric.png`*.

For the credit-card data, several PCA components (V14, V12, V17, V10) correlate strongly with the target — the usable signal there is already encoded in the anonymised features. *Figures:* *`cc_amount_by_class.png`*, *`cc_target_correlation.png`*.

---

4. Feature engineering (Fraud_Data)

Implemented in `src/feature_engineering.py` and `src/geolocation.py`.

4.1 `time_since_signup` — the strongest single signal

Defined as hours between `signup_time` and `purchase_time`.

	Legitimate	Fraud
Median hours to purchase	**1,443**	**0**
Share buying within 1 hour	~0%	**53.7%**

Why it works: real users browse, hesitate, and return days later; automated fraud rings monetise a freshly created account immediately. A near-zero gap is therefore a powerful, intuitive fraud indicator — and being a duration rather than a raw timestamp, it generalises across calendar periods.

4.2 Transaction frequency & velocity

- **`device_user_count`** — distinct accounts per device. Fraud mean **7.15** vs legit **1.12**: classic fraud-ring fingerprint (one device, many accounts).
- **`device_velocity_24h`** — rolling 24-hour purchase count per device. Fraud mean **3.68** vs legit **1.0**. Computed with a vectorized time-based `groupby.rolling` (≈15s on 151k rows vs ≈124s for the naive loop).
- **`device_transaction_count`** — total purchases per device.
- **`user_transaction_count`** — *retained but uninformative*: every `user_id` appears exactly once, so per-user frequency is constant (mean 1.0 for both classes). Flagged honestly; the device-level features carry the signal instead.

4.3 Other time features

`hour_of_day` and `day_of_week` (extracted from `purchase_time`) provide cyclical context for downstream models.

4.4 IP-to-country mapping (geolocation integration)

Three steps in `src/geolocation.py`:

1. **IP → integer** — Fraud_Data stores IPs as floats (e.g. `732758368.79972`); the fractional part is truncated to a 32-bit integer. The helper also accepts dotted-quad strings and returns `NaN` for unparseable values.
2. **Range-based merge** — the lookup table maps **138,846 IP ranges** to 235 countries via lower/upper bounds. A naive join is O(n·m); instead we sort the ranges once and use **binary search (`np.searchsorted`)** to find each transaction's candidate range, then validate the upper bound — **O(n log m)**, running in **~0.6s**. IPs falling in gaps map to `Unknown`.
3. **Result — 85.5%** of transactions matched to a country; the 14.5% `Unknown` (reserved/unallocated ranges) show a *lower* fraud rate (8.6%), so they are not silently driving the signal.

Fraud patterns by country (≥100 transactions):

Country	Transactions	Fraud rate
Ecuador	106	**26.4%**
Tunisia	118	**26.3%**
Peru	119	**26.1%**
Ireland	240	**22.9%**
New Zealand	278	**22.3%**
Saudi Arabia	264	**18.9%**

Several countries run **2–3× the 9.4% baseline**, making country a genuinely useful categorical feature. *Figure: `fraud_rate_by_country.png`.*

4.5 Transformation

A single scikit-learn `ColumnTransformer` (`src/transform.py`): **`StandardScaler`** on numeric features + **`OneHotEncoder(handle_unknown='ignore')`** on categoricals. It is **fit on the training fold only** and reused on the test fold, so no test information leaks into scaling parameters.

---

5. Class-imbalance handling strategy

5.1 Method: SMOTE, on the training set only

The leakage-safe order is enforced: **split first → fit scaler on train → SMOTE on train**. The test set is never resampled, so evaluation reflects real-world prevalence.

5.2 Why SMOTE over the alternatives

Option	Verdict

Option	Verdict
Random undersampling	Rejected as default — at 0.17% fraud (credit-card) it would discard ~99% of legitimate data and the signal it carries.
Random oversampling	Rejected — duplicates minority rows verbatim, encouraging overfitting to those exact points.
SMOTE ✅	Synthesises new minority samples by interpolating between near neighbours, enriching the minority region of feature space without literal duplication.
SMOTENN	Provided as an option for the extreme credit-card imbalance — adds Edited-Nearest-Neighbours cleaning to remove ambiguous synthetic points near the boundary.

5.3 Documented distribution — before vs after (Fraud_Data train fold)

Class	Before	Before %	After SMOTE	After %
0 (legit)	109,568	90.6%	109,568	50.0%
1 (fraud)	11,321	9.4%	109,568	50.0%

Test fold left unchanged at 90.6% / 9.4% — the real-world distribution is preserved for honest evaluation.

---

6. Deliverables produced

- **Cleaned datasets** → `data/processed/fraud_features.csv`, `data/processed/creditcard_clean.csv`.
- **EDA report** → this document + 9 figures in `reports/figures/`.
- **Feature-engineering documentation** → §4 above and `notebooks/feature-engineering.ipynb` (executed).
- **Resampling justification** → §5 above.
- **Tested code** → `src/` modules with 23 passing `pytest` tests and CI.

7. Recommendations for Task 2 (modeling)

1. Lead with **`time_since_signup`**, **`device_user_count`**, **`device_velocity_24h`** and **country** — the features with real separation.
2. Drop or ignore `user_transaction_count` (constant) and treat raw demographic/browser fields as weak priors only.
3. Optimise and threshold-tune on **AUC-PR**; report a precision-recall curve, not accuracy.
4. Compare a Logistic Regression baseline against a tree ensemble (Random Forest / Gradient Boosting) on the SMOTE-balanced train fold.